

Restart the target computer

Request

You can restart the target computer by using the following request format.

Method	Request URI
POST	/api/control/restart

URI parameters

- None

Request headers

- None

Request body

- None

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK

Available device families

- Windows Mobile
- Windows Desktop
- Xbox
- HoloLens
- IoT

Shut down the target computer

Request

You can shut down the target computer by using the following request format.

Method	Request URI
POST	/api/control/shutdown

URI parameters

- None

Request headers

- None

Request body

- None

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- Xbox
- HoloLens
- IoT

Task manager

Start a modern app

Request

You can start a modern app by using the following request format.

Method	Request URI
POST	/api/taskmanager/app

URI parameters

You can specify the following additional parameters on the request URI:

URI parameter	Description
appid	(required) The PRAID for the app you want to start. This value should be hex64 encoded.
package	(required) The full name for the app package you want to start. This value should be hex64 encoded.

Request headers

- None

Request body

- None

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- Xbox
- HoloLens
- IoT

Stop a modern app

Request

You can stop a modern app by using the following request format.

Method	Request URI
DELETE	/api/taskmanager/app

URI parameters

You can specify the following additional parameters on the request URI:

URI parameter	Description
package	(required) The full name of the app packages that you want to stop. This value should be hex64 encoded.
forcestop	(optional) A value of yes indicates that the system should force all processes to stop.

Request headers

- None

Request body

- None

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- Xbox
- HoloLens
- IoT

Kill process by PID

Request

You can kill a process by using the following request format.

Method	Request URI
DELETE	/api/taskmanager/process

URI parameters

You can specify the following additional parameters on the request URI:

URI parameter	Description
pid	(required) The unique process id for the process to stop.

Request headers

- None

Request body

- None

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Desktop
- HoloLens
- IoT

Networking

Get the current IP configuration

Request

You can get the current IP configuration by using the following request format.

Method	Request URI
GET	/api/networking/ipconfig

URI parameters

- None

Request headers

- None

Request body

- None

Response

The response includes the IP configuration in the following template.

JSON

```
{
  "Adapters": [
    {
      "Description": string,
      "HardwareAddress": string,
      "Index": int,
      "Name": string,
      "Type": string,
      "DHCP": {
        "LeaseExpires": int, (timestamp)
        "LeaseObtained": int, (timestamp)
      }
    }
  ]
}
```

```

        "Address": {
            "IpAddress": string,
            "Mask": string
        }
    },
    "WINS": {(WINS is optional)
        "Primary": {
            "IpAddress": string,
            "Mask": string
        },
        "Secondary": {
            "IpAddress": string,
            "Mask": string
        }
    },
    "Gateways": [{ (always 1+)
        "IpAddress": "10.82.128.1",
        "Mask": "255.255.255.255"
    },...
    ],
    "IpAddresses": [{ (always 1+)
        "IpAddress": "10.82.128.148",
        "Mask": "255.255.255.0"
    },...
    ]
},...
]}

```

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- Xbox
- HoloLens
- IoT

Set a static IP address (IPV4 configuration)

Request

Sets the IPV4 configuration with static IP and DNS. If a static IP is not specified, then it enables DHCP. If a static IP is specified, then DNS must be specified also.

Method	Request URI
PUT	/api/networking/ipv4config

URI parameters

URI parameter	Description
AdapterName	(required) The network interface GUID.
IPAddress	The static IP address to set.
SubnetMask	(required if <i>IPAddress</i> is not null) The static subnet mask.
DefaultGateway	(required if <i>IPAddress</i> is not null) The static default gateway.
PrimaryDNS	(required if <i>IPAddress</i> is not null) The static primary DNS to set.
SecondayDNS	(required if <i>PrimaryDNS</i> is not null) The static secondary DNS to set.

For clarity, to set an interface to DHCP, serialize just the `AdapterName` on the wire:

JSON
<pre>{ "AdapterName": "{82F86C1B-2BAE-41E3-B08D-786CA44FEED7}" }</pre>

Request headers

- None

Request body

- None

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
------------------	-------------

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- Xbox
- HoloLens
- IoT

Enumerate wireless network interfaces

Request

You can enumerate the available wireless network interfaces by using the following request format.

Method	Request URI
GET	/api/wifi/interfaces

URI parameters

- None

Request headers

- None

Request body

- None

Response

A list of the available wireless interfaces with details in the following format.

```
JSON
```

```
{
  "Interfaces": [
    {
      "Description": string,
      "GUID": string (guid with curly brackets),
      "Index": int,
      "ProfilesList": [
        {
          "GroupPolicyProfile": bool,
          "Name": string, (Network currently connected to)
          "PerUserProfile": bool
        }, ...
      ]
    }
  ]
}
```

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- Xbox
- HoloLens
- IoT

Enumerate wireless networks

Request

You can enumerate the list of wireless networks on the specified interface by using the following request format.

Method	Request URI
GET	/api/wifi/networks

URI parameters

You can specify the following additional parameters on the request URI:

URI parameter	Description
interface	(required) The GUID for the network interface to use to search for wireless networks, without brackets.

Request headers

- None

Request body

- None

Response

The list of wireless networks found on the provided *interface*. This includes details for the networks in the following format.

JSON

```
{
  "AvailableNetworks": [
    {
      "AlreadyConnected": bool,
      "AuthenticationAlgorithm": string, (WPA2, etc)
      "Channel": int,
      "CipherAlgorithm": string, (for example, AES)
      "Connectable": int, (0 | 1)
      "InfrastructureType": string,
      "ProfileAvailable": bool,
      "ProfileName": string,
      "SSID": string,
      "SecurityEnabled": int, (0 | 1)
      "SignalQuality": int,
      "BSSID": [int,...],
      "PhysicalTypes": [string,...]
    },...
  ]
}
```

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK

HTTP status code	Description
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- Xbox
- HoloLens
- IoT

Connect and disconnect to a Wi-Fi network.

Request

You can connect or disconnect to a Wi-Fi network by using the following request format.

Method	Request URI
POST	/api/wifi/network

URI parameters

You can specify the following additional parameters on the request URI:

URI parameter	Description
interface	(required) The GUID for the network interface you use to connect to the network.
op	(required) Indicates the action to take. Possible values are connect or disconnect.
ssid	(required if <i>op</i> == connect) The SSID to connect to.
key	(required if <i>op</i> == connect and network requires authentication) The shared key.
createprofile	(required) Create a profile for the network on the device. This will cause the device to auto-connect to the network in the future. This can be yes or no .

Request headers

- None

Request body

- None

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK

Available device families

- Windows Mobile
- Windows Desktop
- Xbox
- HoloLens
- IoT

Delete a Wi-Fi profile

Request

You can delete a profile associated with a network on a specific interface by using the following request format.

Method	Request URI
DELETE	/api/wifi/profile

URI parameters

You can specify the following additional parameters on the request URI:

URI parameter	Description
interface	(required) The GUID for the network interface associated with the profile to delete.
profile	(required) The name of the profile to delete.

Request headers

- None

Request body

- None

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK

Available device families

- Windows Mobile
- Windows Desktop
- Xbox
- HoloLens
- IoT

Windows Error Reporting (WER)

Download a Windows error reporting (WER) file

Request

You can download a WER-related file by using the following request format.

Method	Request URI
GET	/api/wer/report/file

URI parameters

You can specify the following additional parameters on the request URI:

URI parameter	Description
user	(required) The user name associated with the report.
type	(required) The type of report. This can be either queried or archived .
name	(required) The name of the report. This should be base64 encoded.
file	(required) The name of the file to download from the report. This should be base64 encoded.

Request headers

- None

Request body

- None

Response

- Response contains the requested file.

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Desktop
- HoloLens
- IoT

Enumerate files in a Windows error reporting (WER) report

Request

You can enumerate the files in a WER report by using the following request format.

Method	Request URI
GET	/api/wer/report/files

URI parameters

You can specify the following additional parameters on the request URI:

URI parameter	Description
user	(required) The user associated with the report.
type	(required) The type of report. This can be either queried or archived .
name	(required) The name of the report. This should be base64 encoded.

Request headers

- None

Request body

JSON

```
{  
  "Files": [  
    {  
      "Name": string, (Filename, not base64 encoded)  
      "Size": int (bytes)  
    }, ...  
  ]  
}
```

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Desktop
- HoloLens
- IoT

List the Windows error reporting (WER) reports

Request

You can get the WER reports by using the following request format.

Method	Request URI
GET	/api/wer/reports

URI parameters

- None

Request headers

- None

Request body

- None

Response

The WER reports in the following format.

JSON

```
{
  "WerReports": [
    {
      "User": string,
      "Reports": [
        {
          "CreationTime": int,
          "Name": string, (not base64 encoded)
          "Type": string ("Queue" or "Archive")
        },
        ...
      ]
    },
    ...
  ]
}
```

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Desktop
- HoloLens
- IoT

Windows Performance Recorder (WPR)

Start tracing with a custom profile

Request

You can upload a WPR profile and start tracing using that profile by using the following request format. Only one trace can run at a time. The profile will not remain on the device.

Method	Request URI
POST	/api/wpr/customtrace

URI parameters

- None

Request headers

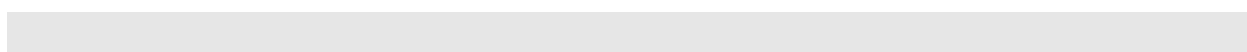
- None

Request body

- A multi-part conforming http body that contains the custom WPR profile.

Response

The WPR session status in the following format.



JSON

```
{
  "SessionType": string, (Running or Idle)
  "State": string (normal or boot)
}
```

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- IoT

Start a boot performance tracing session

Request

You can start a boot WPR tracing session by using the following request format. This is also known as a performance tracing session.

Method	Request URI
POST	/api/wpr/boottrace

URI parameters

You can specify the following additional parameters on the request URI:

URI parameter	Description
---------------	-------------

URI parameter	Description
profile	(required) This parameter is required on start. The name of the profile that should start a performance tracing session. The possible profiles are stored in perfprofiles/profiles.json.

Request headers

- None

Request body

- None

Response

On start, this API returns the WPR session status in the following format.

JSON
<pre>{ "SessionType": string, (Running or Idle) "State": string (boot) }</pre>

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- IoT

Stop a boot performance tracing session

Request

You can stop a boot WPR tracing session by using the following request format. This is also known as a performance tracing session.

Method	Request URI
GET	/api/wpr/boottrace

URI parameters

- None

Request headers

- None

Request body

- None

Response

- None. **Note:** This is a long running operation. It will return when the ETL is finished writing to disk.

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- IoT

Start a performance tracing session

Request

You can start a WPR tracing session by using the following request format. This is also known as a performance tracing session. Only one trace can run at a time.

Method	Request URI
POST	/api/wpr/trace

URI parameters

You can specify the following additional parameters on the request URI:

URI parameter	Description
profile	(required) The name of the profile that should start a performance tracing session. The possible profiles are stored in perfprofiles/profiles.json.

Request headers

- None

Request body

- None

Response

On start, this API returns the WPR session status in the following format.

JSON
<pre>{ "SessionType": string, (Running or Idle) "State": string (normal) }</pre>

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes

HTTP status code	Description
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- IoT

Stop a performance tracing session

Request

You can stop a WPR tracing session by using the following request format. This is also known as a performance tracing session.

Method	Request URI
GET	/api/wpr/trace

URI parameters

- None

Request headers

- None

Request body

- None

Response

- None. **Note:** This is a long running operation. It will return when the ETL is finished writing to disk.

Status code

This API has the following expected status codes.

HTTP status code	Description
------------------	-------------

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- IoT

Retrieve the status of a tracing session

Request

You can retrieve the status of the current WPR session by using the following request format.

Method	Request URI
GET	/api/wpr/status

URI parameters

- None

Request headers

- None

Request body

- None

Response

The status of the WPR tracing session in the following format.

JSON

```
{  
  "SessionType": string, (Running or Idle)
```



```
"State": string (normal or boot)
}
```

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- IoT

List completed tracing sessions (ETLs)

Request

You can get a listing of ETL traces on the device using the following request format.

Method	Request URI
GET	/api/wpr/tracefiles

URI parameters

- None

Request headers

- None

Request body

- None

Response

The listing of completed tracing sessions is provided in the following format.

JSON

```
{
  "Items": [
    {
      "CurrentDir": string (filepath),
      "DateCreated": int (File CreationTime),
      "FileSize": int (bytes),
      "Id": string (filename),
      "Name": string (filename),
      "SubPath": string (filepath),
      "Type": int
    }
  ]
}
```

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- IoT

Download a tracing session (ETL)

Request

You can download a tracefile (boot trace or user-mode trace) using the following request format.

Method	Request URI
GET	/api/wpr/tracefile

URI parameters

You can specify the following additional parameter on the request URI:

URI parameter	Description
filename	(required) The name of the ETL trace to download. These can be found in /api/wpr/tracefiles

Request headers

- None

Request body

- None

Response

- Returns the trace ETL file.

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- IoT

Delete a tracing session (ETL)

Request

You can delete a tracefile (boot trace or user-mode trace) using the following request format.

Method	Request URI
DELETE	/api/wpr/tracefile

URI parameters

You can specify the following additional parameter on the request URI:

URI parameter	Description
filename	(required) The name of the ETL trace to delete. These can be found in /api/wpr/tracefiles

Request headers

- None

Request body

- None

Response

- Returns the trace ETL file.

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- IoT

DNS-SD Tags

View Tags

Request

View the currently applied tags for the device. These are advertised via DNS-SD TXT records in the T key.

Method	Request URI
GET	/api/dns-sd/tags

URI parameters

- None

Request headers

- None

Request body

- None

Response The currently applied tags in the following format.

JSON
<pre>{ "tags": ["tag1", "tag2", ...] }</pre>

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
5XX	Server Error

Available device families

- Windows Mobile
 - Windows Desktop
 - Xbox
 - HoloLens
 - IoT
-

Delete Tags

Request

Delete all tags currently advertised by DNS-SD.

Method	Request URI
DELETE	/api/dns-sd/tags

URI parameters

- None

Request headers

- None

Request body

- None

Response

- None

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
5XX	Server Error

Available device families

- Windows Mobile
- Windows Desktop

- Xbox
- HoloLens
- IoT

Delete Tag

Request

Delete a tag currently advertised by DNS-SD.

Method	Request URI
DELETE	/api/dns-sd/tag

URI parameters

URI parameter	Description
tagValue	(required) The tag to be removed.

Request headers

- None

Request body

- None

Response

- None

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK

Available device families

- Windows Mobile
- Windows Desktop
- Xbox

- HoloLens
- IoT

Add a Tag

Request

Add a tag to the DNS-SD advertisement.

Method	Request URI
POST	/api/dns-sd/tag

URI parameters

URI parameter	Description
tagValue	(required) The tag to be added.

Request headers

- None

Request body

- None

Response

- None

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
401	Tag space Overflow. Results when the proposed tag is too long for the resulting DNS-SD service record.

Available device families

- Windows Mobile

- Windows Desktop
- Xbox
- HoloLens
- IoT

App File Explorer

Get known folders

Request

Obtain a list of accessible top-level folders.

Method	Request URI
GET	/api/filesystem/apps/knownfolders

URI parameters

- None

Request headers

- None

Request body

- None

Response The available folders in the following format.

JSON

```
{ "KnownFolders": [  
  "folder0",  
  "folder1", ...  
]}
```

Status code

This API has the following expected status codes.

HTTP status code	Description
------------------	-------------

HTTP status code	Description
200	Deploy request accepted and being processed
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- Xbox
- IoT

Get files

Request

Obtain a list of files in a folder.

Method	Request URI
GET	/api/filesystem/apps/files

URI parameters

URI parameter	Description
knownfolderid	(required) The top-level directory where you want the list of files. Use LocalAppData for access to sideloaded apps.
packagefullname	(required if <i>knownfolderid</i> == LocalAppData) The package full name of the app you are interested in.
path	(optional) The sub-directory within the folder or package specified above.

Request headers

- None

Request body

- None

Response The available folders in the following format.

JSON

```
{  
  "Items": [  
    {  
      "CurrentDir": string (folder under the requested known folder),  
      "DateCreated": int,  
      "FileSize": int (bytes),  
      "Id": string,  
      "Name": string,  
      "SubPath": string (present if this item is a folder, this is the  
name of the folder),  
      "Type": int  
    },...  
  ]  
}
```

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- Xbox
- IoT

Download a file

Request

Obtain a file from a known folder or appLocalData.

Method	Request URI
GET	/api/filesystem/apps/file

URI parameters

URI parameter	Description
knownfolderid	(required) The top-level directory where you want to download files. Use LocalAppData for access to sideloaded apps.
filename	(required) The name of the file being downloaded.
packagefullname	(required if <i>knownfolderid</i> == LocalAppData) The package full name you are interested in.
path	(optional) The sub-directory within the folder or package specified above.

Request headers

- None

Request body

- The file requested, if present

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
200	The requested file
404	File not found
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- Xbox
- IoT

Rename a file

Request

Rename a file in a folder.

Method	Request URI
POST	/api/filesystem/apps/rename

URI parameters

URI parameter	Description
knownfolderid	(required) The top-level directory where the file is located. Use LocalAppData for access to sideloaded apps.
filename	(required) The original name of the file being renamed.
newfilename	(required) The new name of the file.
packagefullname	(required if <i>knownfolderid</i> == LocalAppData) The package full name of the app you are interested in.
path	(optional) The sub-directory within the folder or package specified above.

Request headers

- None

Request body

- None

Response

- None

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK. The file is renamed
404	File not found
5XX	Error codes

Available device families

- Windows Mobile

- Windows Desktop
- HoloLens
- Xbox
- IoT

Delete a file

Request

Delete a file in a folder.

Method	Request URI
DELETE	/api/filesystem/apps/file

URI parameters

URI parameter	Description
knownfolderid	(required) The top-level directory where you want to delete files. Use LocalAppData for access to sideloaded apps.
filename	(required) The name of the file being deleted.
packagefullname	(required if <i>knownfolderid</i> == LocalAppData) The package full name of the app you are interested in.
path	(optional) The sub-directory within the folder or package specified above.

Request headers

- None

Request body

- None

Response

- None

Status code

This API has the following expected status codes.

HTTP status code	Description
------------------	-------------

HTTP status code	Description
200	OK. The file is deleted
404	File not found
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- Xbox
- IoT

Upload a file

Request

Upload a file to a folder. This will overwrite an existing file with the same name, but will not create new folders.

Method	Request URI
POST	/api/filesystem/apps/file

URI parameters

URI parameter	Description
knownfolderid	(required) The top-level directory where you want to upload files. Use LocalAppData for access to sideloaded apps.
packagefullname	(required if <i>knownfolderid</i> == LocalAppData) The package full name of the app you are interested in.
extract	(required) True or false. This indicates whether the file should be extracted after upload.
path	(optional) The sub-directory within the folder or package specified above.

Request headers

- None

Request body

- None

Response

Status code

This API has the following expected status codes.

HTTP status code	Description
200	OK. The file is uploaded
4XX	Error codes
5XX	Error codes

Available device families

- Windows Mobile
- Windows Desktop
- HoloLens
- Xbox
- IoT

Write a custom plugin for Windows Device Portal

Article • 10/20/2022

Learn how to write a UWP app that uses the Windows Device Portal (WDP) to host a web page and provide diagnostic information.

Starting with Windows 10 Creators Update (version 1703, build 15063), you can use Device Portal to host your app's diagnostic interfaces. This article covers the three pieces needed to create a DevicePortalProvider for your app – the [application package manifest](#) changes, setting up your app's connection to the [Device Portal service](#), and handling an incoming request.

Create a new UWP app project

In Microsoft Visual Studio, create a new UWP app project. Go to **File > New > Project** and select **Blank App (Windows Universal) for C#**, and then click **Next**. In the **Configure your new project** dialog box. Name the project "DevicePortalProvider" and then click **Create**. This will be the app that contains the app service. You may need to update Visual Studio or install the latest [Windows SDK](#).

Add the devicePortalProvider extension to your application package manifest

You will need to add some code to your *package.appxmanifest* file in order to make your app functional as a Device Portal plugin. First, add the following namespace definitions at the top of the file. Also add them to the `IgnorableNamespaces` attribute.

XML

```
<Package
...

xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows10/restrictedcapabilities"
xmlns:uap4="http://schemas.microsoft.com/appx/manifest/uap/windows10/4"
IgnorableNamespaces="uap mp rescap uap4">
...
```

In order to declare that your app is a Device Portal Provider, you need to create an app service and a new Device Portal Provider extension that uses it. Add both the `windows.appService` extension and the `windows.devicePortalProvider` extension in the `Extensions` element under `Application`. Make sure the `AppServiceName` attributes match in each extension. This indicates to the Device Portal service that this app service can be launched to handle requests on the handler namespace.

XML

```
...
<Application
  Id="App"
  Executable="$targetnametoken$.exe"
  EntryPoint="DevicePortalProvider.App">
  ...
  <Extensions>
    <uap:Extension Category="windows.appService"
      EntryPoint="MySampleProvider.SampleProvider">
      <uap:AppService Name="com.sampleProvider.wdp" />
    </uap:Extension>
    <uap4:Extension Category="windows.devicePortalProvider">
      <uap4:DevicePortalProvider
        DisplayName="My Device Portal Provider Sample App"
        AppServiceName="com.sampleProvider.wdp"
        HandlerRoute="/MyNamespace/api/" />
      </uap4:Extension>
    </Extensions>
  </Application>
  ...
```

The `HandlerRoute` attribute references the REST namespace claimed by your app. Any HTTP requests on that namespace (implicitly followed by a wildcard) received by the Device Portal service will be sent to your app to be handled. In this case, any successfully authenticated HTTP request to `<ip_address>/MyNamespace/api/*` will be sent to your app. Conflicts between handler routes are settled via a "longest wins" check: whichever route matches more of the requests is selected, meaning that a request to `"/MyNamespace/api/foo"` will match against a provider with `"/MyNamespace/api"` rather than one with `"/MyNamespace"`.

Two new capabilities are required for this functionality. they must also be added to your *package.appxmanifest* file.

XML

```
...
<Capabilities>
  ...
```

```
<Capability Name="privateNetworkClientServer" />
<rescap:Capability Name="devicePortalProvider" />
</Capabilities>
...
```

ⓘ Note

The capability "devicePortalProvider" is restricted ("rescap"), which means you must get prior approval from the Store before your app can be published there. However, this does not prevent you from testing your app locally through sideloading. For more information about restricted capabilities, see [App capability declarations](#).

Set up your background task and WinRT Component

In order to set up the Device Portal connection, your app must hook up an app service connection from the Device Portal service with the instance of Device Portal running within your app. To do this, add a new WinRT Component to your application with a class that implements [IBackgroundTask](#).

C#

```
using Windows.System.Diagnostics.DevicePortal;
using Windows.ApplicationModel.Background;

namespace MySampleProvider {
    // Implementing a DevicePortalConnection in a background task
    public sealed class SampleProvider : IBackgroundTask {
        BackgroundTaskDeferral taskDeferral;
        DevicePortalConnection devicePortalConnection;
        //...
    }
}
```

Make sure that its name matches the namespace and class name set up by the AppService EntryPoint ("MySampleProvider.SampleProvider"). When you make your first request to your Device Portal provider, Device Portal will stash the request, launch your app's background task, call its **Run** method, and pass in an [IBackgroundTaskInstance](#). Your app then uses it to set up a [DevicePortalConnection](#) instance.

C#

```
// Implement background task handler with a DevicePortalConnection
public void Run(IBackgroundTaskInstance taskInstance) {
    // Take a deferral to allow the background task to continue executing
    this.taskDeferral = taskInstance.GetDeferral();
    taskInstance.Canceled += TaskInstance_Canceled;

    // Create a DevicePortal client from an AppServiceConnection
    var details = taskInstance.TriggerDetails as AppServiceTriggerDetails;
    var appServiceConnection = details.AppServiceConnection;
    this.devicePortalConnection =
        DevicePortalConnection.GetForAppServiceConnection(appServiceConnection);

    // Add Closed, RequestReceived handlers
    devicePortalConnection.Closed += DevicePortalConnection_Closed;
    devicePortalConnection.RequestReceived +=
        DevicePortalConnection_RequestReceived;
}
```

There are two events that must be handled by the app to complete the request handling loop: **Closed**, for whenever the Device Portal service shuts down, and **RequestReceived**, which surfaces incoming HTTP requests and provides the main functionality of the Device Portal provider.

Handle the RequestReceived event

The **RequestReceived** event will be raised once for every HTTP request that is made on your plugin's specified Handler Route. The request handling loop for Device Portal providers is similar to that in NodeJS Express: the request and response objects are provided together with the event, and the handler responds by filling in the response object. In Device Portal providers, the **RequestReceived** event and its handlers use **Windows.Web.Http.HttpRequestMessage** and **HttpResponseMessage** objects.

C#

```
// Sample RequestReceived echo handler: respond with an HTML page including
// the query and some additional process information.
private void DevicePortalConnection_RequestReceived(DevicePortalConnection
sender, DevicePortalConnectionRequestReceivedEventArgs args)
{
    var req = args.RequestMessage;
    var res = args.ResponseMessage;

    if (req.RequestUri.AbsolutePath.EndsWith("/echo"))
    {
        // construct an html response message
        string con = "<h1>" + req.RequestUri.AbsoluteUri + "</h1><br/>";
        var proc =
            Windows.System.Diagnostics.ProcessDiagnosticInfo.GetForCurrentProcess();
    }
}
```

```

        con += String.Format("This process is consuming {0} bytes (Working
Set)<br/>", proc.MemoryUsage.GetReport().WorkingSetSizeInBytes);
        con += String.Format("The process PID is {0}<br/>", proc.ProcessId);
        con += String.Format("The executable filename is {0}",
proc.ExecutableFileName);
        res.Content = new Windows.Web.HttpStringContent(con);
        res.Content.Headers.ContentType = new
Windows.Web.Http.Headers.HttpMediaTypeHeaderValue("text/html");
        res.StatusCode = Windows.Web.Http.HttpStatusCode.Ok;
    }
    //...
}

```

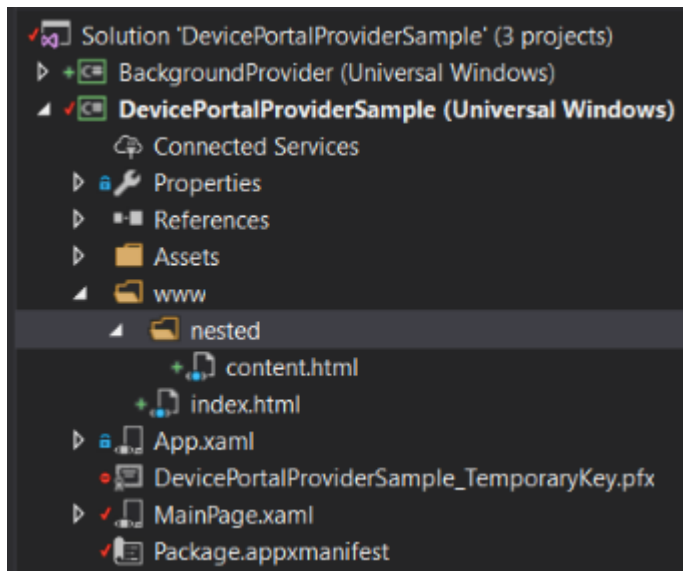
In this sample request handler, we first pull the request and response objects out of the *args* parameter, then create a string with the request URL and some additional HTML formatting. This is added into the Response object as an **HttpStringContent** instance. Other **IHttpContent** classes, such as those for "String" and "Buffer," are also allowed.

The response is then set as an HTTP response and given a 200 (OK) status code. It should render as expected in the browser that made the original call. Note that when the **RequestReceived** event handler returns, the response message is automatically returned to the user agent: no additional "send" method is needed.



Providing static content

Static content can be served directly from a folder within your package, making it very easy to add a UI to your provider. The easiest way to serve static content is to create a content folder in your project that can map to a URL.

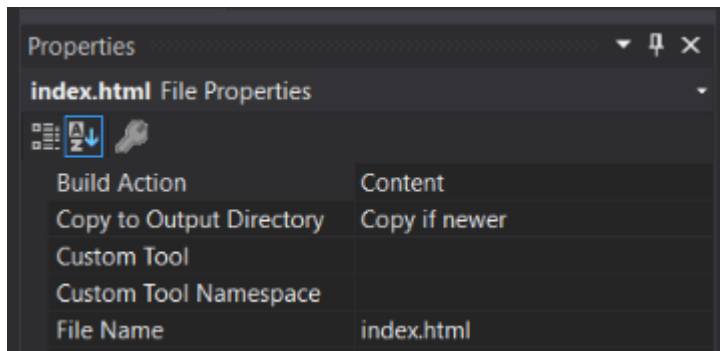


Then, add a route handler in your **RequestReceived** event handler that detects static content routes and maps a request appropriately.

C#

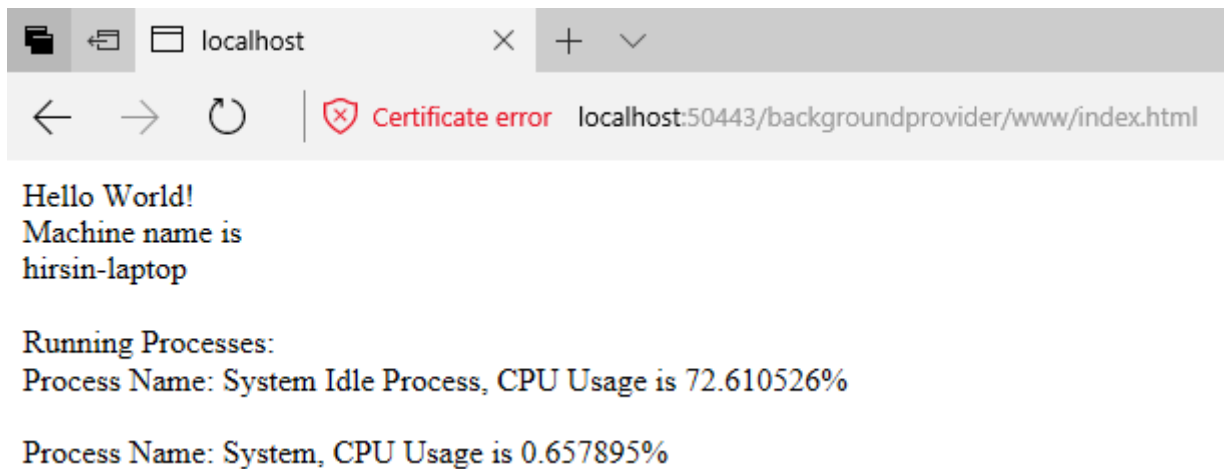
```
if (req.RequestUri.LocalPath.ToLower().Contains("/www/")) {
    var filePath = req.RequestUri.AbsolutePath.Replace('/', '\\').ToLower();
    filePath = filePath.Replace("\\backgroundprovider", "")
    try {
        var fileStream =
            Windows.ApplicationModel.Package.Current.InstalledLocation.OpenStreamForRead
            Async(filePath).GetAwaiter().GetResult();
        res.StatusCode = HttpStatusCode.Ok;
        res.Content = new HttpStreamContent(fileStream.AsInputStream());
        res.Content.Headers.ContentType = new
            HttpMediaTypeHeaderValue("text/html");
    } catch (FileNotFoundException e) {
        string con = String.Format("<h1>{0} - not found</h1>\r\n",
            filePath);
        con += "Exception: " + e.ToString();
        res.Content = new Windows.Web.Http.HttpStringContent(con);
        res.StatusCode = Windows.Web.Http.HttpStatusCode.NotFound;
        res.Content.Headers.ContentType = new
            Windows.Web.Http.Headers.HttpMediaTypeHeaderValue("text/html");
    }
}
```

Make sure that all files inside of the content folder are marked as "Content" and set to "Copy if newer" or "Copy always" in Visual Studio's Properties menu. This ensures that the files will be inside your AppX Package when you deploy it.



Using existing Device Portal resources and APIs

Static content served by a Device Portal provider is served on the same port as the core Device Portal service. This means that you can reference the existing JS and CSS included with Device Portal with simple `<link>` and `<script>` tags in your HTML. In general, we suggest the use of *rest.js*, which wraps all the core Device Portal REST APIs in a convenient *webbRest* object, and the *common.css* file, which will allow you to style your content to fit with the rest of Device Portal's UI. You can see an example of this in the *index.html* page included in the sample. It uses *rest.js* to retrieve the device name and running processes from Device Portal.



Importantly, use of the `HttpPost/DeleteExpect200` methods on *webbRest* will automatically do the [CSRF handling](#) for you, which allows your webpage to call state-changing REST APIs.

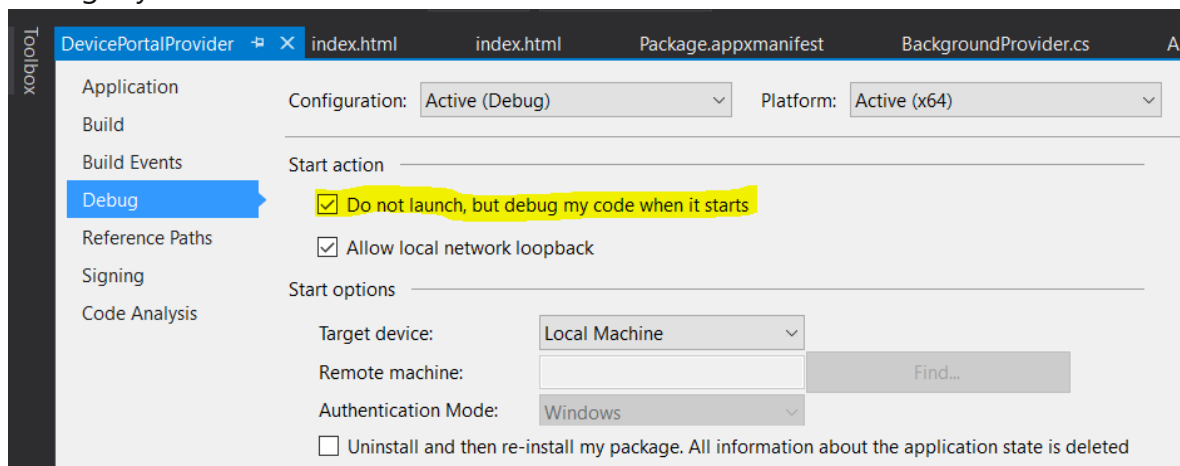
ⓘ Note

The static content included with Device Portal does not come with a guarantee against breaking changes. While the APIs are not expected to change often, they may, especially in the *common.js* and *controls.js* files, which your provider should not use.

Debugging the Device Portal connection

In order to debug your background task, you must change the way Visual Studio runs your code. Follow the steps below for debugging an app service connection to inspect how your provider is handling the HTTP requests:

1. From the Debug menu, select DevicePortalProvider Properties.
2. Under the Debugging tab, in the Start action section, select "Do not launch, but debug my code when it starts".



3. Set a breakpoint in your RequestReceived handler function.



ⓘ Note

Make sure the build architecture matches the architecture of the target exactly. If you are using a 64-bit PC, you must deploy using an AMD64 build. 4. Press F5 to deploy your app 5. Turn Device Portal off, then turn it back on so that it finds your app (only needed when you change your app manifest – the rest of the time you can simply re-deploy and skip this step). 6. In your browser, access the provider's namespace, and the breakpoint should be hit.

Related topics

- [Windows Device Portal overview](#)
- [Create and consume an app service](#)

Provision Windows Device Portal with a custom SSL certificate

Article • 10/20/2022

Windows Device Portal (WDP) provides a way for device administrators to install a custom certificate for use in HTTPS communication.

While you can do this on your own PC, this feature is primarily intended for enterprises that have an existing certificate infrastructure in place.

For example, a company might have a certificate authority (CA) that it uses to sign certificates for intranet websites served over HTTPS. This feature stands on top of that infrastructure.

Overview

By default, WDP generates a self-signed root CA, and then uses that to sign SSL certificates for every endpoint it is listening on. This includes `localhost`, `127.0.0.1`, and `::1` (IPv6 localhost).

Also included are the device's hostname (for example, `https://LivingRoomPC`) and each link-local IP address assigned to the device (up to two [IPv4, IPv6] per network adaptor). You can see the link-local IP addresses for a device by looking at the Networking tool in the WDP. They'll start with `10.` or `192.` for IPv4, or `fe80:` for IPv6.

In the default setup, a certificate warning may appear in your browser because of the untrusted root CA. Specifically, the SSL cert provided by WDP is signed by a root CA that the browser or PC doesn't trust. This can be fixed by creating a new trusted root CA.

Create a root CA

This should only be done if your company (or home) doesn't have a certificate infrastructure set up, and should only be done once. The following PowerShell script creates a root CA called *WdpTestCA.cer*. Installing this file to the local machine's Trusted Root Certification Authorities will cause your device to trust SSL certs that are signed by this root CA. You can (and should) install this .cer file on each PC that you want to connect to WDP.

PowerShell

```

$CN = "PickAName"
$OutputPath = "c:\temp\"

# Create root certificate authority
$FilePath = $OutputPath + "WdpTestCA.cer"
$Subject = "CN="+$CN
$rootCA = New-SelfSignedCertificate -certstorelocation cert:\currentuser\my
-Subject $Subject -HashAlgorithm "SHA512" -KeyUsage CertSign,CRLSign
$rootCAFile = Export-Certificate -Cert $rootCA -FilePath $FilePath

```

Once this is created, you can use the *WdpTestCA.cer* file to sign SSL certs.

Create an SSL certificate with the root CA

SSL certificates have two critical functions: securing your connection through encryption and verifying that you are actually communicating with the address displayed in the browser bar (Bing.com, 192.168.1.37, etc.) and not a malicious third party.

The following PowerShell script creates an SSL certificate for the `localhost` endpoint. Each endpoint that WDP listens on needs its own certificate; you can replace the `$IssuedTo` argument in the script with each of the different endpoints for your device: the hostname, localhost, and the IP addresses.

PowerShell

```

$IssuedTo = "localhost"
$Password = "PickAPassword"
$OutputPath = "c:\temp\"
$rootCA = Import-Certificate -FilePath C:\temp\WdpTestCA.cer -
CertStoreLocation Cert:\CurrentUser\My\

# Create SSL cert signed by certificate authority
$IssuedToClean = $IssuedTo.Replace(":", "-").Replace(" ", "_")
$FilePath = $OutputPath + $IssuedToClean + ".pfx"
$Subject = "CN="+$IssuedTo
$cert = New-SelfSignedCertificate -certstorelocation cert:\localmachine\my -
Subject $Subject -DnsName $IssuedTo -Signer $rootCA -HashAlgorithm "SHA512"
$certFile = Export-PfxCertificate -cert $cert -FilePath $FilePath -Password
(ConvertTo-SecureString -String $Password -Force -AsPlainText)

```

If you have multiple devices, you can reuse the localhost .pfx files, but you'll still need to create IP address and hostname certificates for each device separately.

When the bundle of .pfx files is generated, you will need to load them into the WDP.

Provision Windows Device Portal with the certification(s)

For each .pfx file that you've created for a device, you'll need to run the following command from an elevated command prompt.

Windows Command Prompt

```
WebManagement.exe -SetCert <Path to .pfx file> <password for pfx>
```

See below for example usage:

Windows Command Prompt

```
WebManagement.exe -SetCert localhost.pfx PickAPassword
WebManagement.exe -SetCert --1.pfx PickAPassword
WebManagement.exe -SetCert MyLivingRoomPC.pfx PickAPassword
```

Once you have installed the certificates, simply restart the service so the changes can take effect:

Windows Command Prompt

```
sc stop webmanagement
sc start webmanagement
```

Tip

IP addresses can change over time. Many networks use DHCP to give out IP addresses, so devices don't always get the same IP address they had previously. If you've created a certificate for an IP address on a device and that device's address has changed, WDP will generate a new certificate using the existing self-signed cert, and it will stop using the one you created. This will cause the cert warning page to appear in your browser again. For this reason, we recommend connecting to your devices through their hostnames, which you can set in the Windows Device Portal. These will stay the same regardless of IP addresses.

Windows App Certification Kit

Article • 10/24/2022

To get your app [Windows Certified](#) or prepare it for [publication to the Microsoft Store](#), you should validate and test it locally first. This topic shows you how to install and run the [Windows App Certification Kit](#) to ensure your app is safe and efficient.

Prerequisites

Prerequisites for testing a Universal Windows app:

- You must install and run Windows 10.
- You must install the [Windows App Certification Kit](#), which is included in the Windows Software Development Kit (SDK) for Windows 10.
- You must [enable your device for development](#).
- You must deploy the Windows app that you want to test to your computer.

ⓘ Note

In-place upgrades: Installing a more recent [Windows App Certification Kit](#) will replace any previously installed version of the kit.

What's new

Tests for Windows [Desktop Bridge Apps](#) are now supported in the kit. [Windows Desktop Bridge app tests](#) can give your app the best chance of being published on Microsoft Store or get certified.

The kit can now be integrated into an automated testing where no interactive user session is available.

The App Prelaunch Validation test is no longer supported.

Known issues

The following is a list of known issues with the Windows App Certification Kit:

During testing, if an installer terminates but leaves active processes or windows running, the app certification kit may detect that there is still work to be done by the installer. In

this case, the kit appears stuck running the "Process Install Trace Files" task and it's not possible to move forward with the UI.

Resolution: After your installer is complete, manually close any active processes or windows spawned by the installer.

For Arm UWA, or any UWA app that doesn't target the device family desktop or OneCore, a message may appear in the final report that states "Not all tests were run during validation. This may impact your Store submission.". This message does not apply in cases where the user didn't manually deselect tests.

Resolution: n/a

For Desktop Bridge Apps using Windows SDK Version 10.0.15063 please ignore any failures in Application Manifest Resources test that flag your image not confirming to the expected dimensions if those dimensions are only off by one pixel. The test is supposed to have a +/-1 pixel tolerance. E.g. A small tile at 125% would be 88.75x88.75px if rounded up to 89x89px this would fail the size restrictions of 88x88px.

Resolution: n/a

Validate your Windows app using the Windows App Certification Kit interactively

1. From the **Start** menu, search **Apps**, find **Windows Kits**, and click **Windows App Cert Kit**.
2. From the Windows App Certification Kit, select the category of validation you would like to perform. For example: If you are validating a Windows app, select **Validate a Windows app**.

You may browse directly to the app you're testing, or choose the app from a list in the UI. When the Windows App Certification Kit is run for the first time, the UI lists all the Windows apps that you have installed on your computer. For any subsequent runs, the UI will display the most recent Windows apps that you have validated. If the app that you want to test is not listed, you can click on **My app isn't listed** to get a comprehensive list of all apps installed on your system.

3. After you have input or selected the app that you want to test, click **Next**.
4. From the next screen, you will see the test workflow that aligns to the app type you are testing. If a test is grayed out in the list, the test is not applicable to your environment. For example, if you are testing a Windows 10 app on Windows 7,

only static tests will apply to the workflow. Note that the Microsoft Store may apply all tests from this workflow. Select the tests you want to run and click **Next**.

The Windows App Certification Kit begins validating the app.

5. At the prompt after the test, enter the path to the folder where you want to save the test report.

The Windows App Certification Kit creates an HTML along with an XML report and saves it in this folder.

6. Open the report file and review the results of the test.

ⓘ Note

If you're using Visual Studio, you can run the Windows App Certification Kit when you create your app package. See [Packaging UWP apps](#) to learn how.

Validate your Windows app using the Windows App Certification Kit from a command line

ⓘ Important

The Windows App Certification Kit must be run within the context of an active user session.

1. Open a command window with admin rights ("Run as administrator").
2. In the command window, navigate to the directory that contains the Windows App Certification Kit.

Note The default path is C:\Program Files (x86)\Windows Kits\10\App Certification Kit\.

3. Enter the following commands in this order to test an app that is already installed on your test computer:

```
appcert.exe reset
```

```
appcert.exe test -packagefullname [package full name] -reportoutputpath  
[report file name]
```

Or you can use the following commands if the app is not installed. The Windows App Certification Kit will open the package and apply the appropriate test workflow:

```
appcert.exe reset
```

```
appcert.exe test -appxpackagepath [package path] -reportoutputpath [report  
file name]
```

4. After the test completes, open the report file named [report file name] and review the test results.

Note The Windows App Certification Kit can be run from a service, but the service must initiate the kit process within an active user session and cannot be run in Session0.

Note For more info about the Windows App Certification Kit command line, enter the command `appcert.exe /?`

Testing with a low-power computer

The performance test thresholds of the Windows App Certification Kit are based on the performance of a low-power computer.

The characteristics of the computer on which the test is performed can influence the test results. To determine if your app's performance meets the [Microsoft Store Policies](#), we recommend that you test your app on a low-power computer, such as an Intel Atom processor-based computer with a screen resolution of 1366x768 (or higher) and a rotational hard drive (as opposed to a solid-state hard drive).

As low-power computers evolve, their performance characteristics might change over time. Refer to the most current [Microsoft Store Policies](#) and test your app with the most current version of the Windows App Certification Kit to make sure that your app complies with the latest performance requirements.

Related topics

- [Using the Windows App Certification Kit](#)
 - [Certification requirements for Windows Desktop Apps](#)
 - [Windows App Certification Kit tests](#)
 - [Microsoft Store Policies](#)
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Windows App Certification Kit tests

Article • 03/29/2023

The [Windows App Certification Kit](#) contains a number of tests that help ensure your app is ready to be published to the Microsoft Store. The tests are listed below with their criteria, details, and suggested actions in the case of failure.

Deployment and launch tests

Monitors the app during certification testing to record when it crashes or hangs.

Background

Apps that stop responding or crash can cause the user to lose data and have a poor experience.

We expect apps to be fully functional without the use of Windows compatibility modes, AppHelp messages, or compatibility fixes.

Apps must not list DLLs to load in the HKEY-LOCAL-MACHINE\Software\Microsoft\Windows NT\CurrentVersion\Windows\ApplInit-DLLs registry key.

Test details

We test the app resilience and stability throughout the certification testing.

The Windows App Certification Kit calls

[IApplicationActivationManager::ActivateApplication](#) to launch apps. For **ActivateApplication** to launch an app, User Account Control (UAC) must be enabled and the screen resolution must be at least 1024 x 768 or 768 x 1024. If either condition is not met, your app will fail this test.

Corrective actions

Make sure UAC is enabled on the test computer.

Make sure you are running the test on a computer with large enough screen.

If your app fails to launch and your test platform satisfies the prerequisites of [ActivateApplication](#), you can troubleshoot the problem by reviewing the activation

event log. To find these entries in the event log:

1. Open eventvwr.exe and navigate to the Application and Services Log\Microsoft\Windows\Immersive-Shell folder.
2. Filter the view to show Event Ids: 5900-6000.
3. Review the log entries for info that might explain why the app didn't launch.

Troubleshoot the file with the problem, identify and fix the problem. Rebuild and re-test the app. You can also check if a dump file was generated in the Windows App Certification Kit log folder that can be used to debug your app.

Platform Version Launch test

Checks that the Windows app can run on a future version of the OS. This test has historically been only applied to the Desktop app workflow, but this is now enabled for the Store and Universal Windows Platform (UWP) workflows.

Background

Operating system version info has restricted usage for the Microsoft Store. This has often been incorrectly used by apps to check OS version so that the app can provide users with functionality that is specific to an OS version.

Test details

The Windows App Certification Kit uses the HighVersionLie to detect how the app checks the OS version. If the app crashes, it will fail this test.

Corrective action

Apps should use Version API helper functions to check this. See [Operating System Version](#) for more information.

Background tasks cancellation handler validation

This verifies that the app has a cancellation handler for declared background tasks. There needs to be a dedicated function that will be called when the task is cancelled. This test is applied only for deployed apps.

Background

Store apps can register a process that runs in the background. For example, an email app may ping a server from time to time. However, if the OS needs these resources, it will cancel the background task, and apps should gracefully handle this cancellation. Apps that don't have a cancellation handler may crash or not close when the user tries to close the app.

Test details

The app is launched, suspended and the non-background portion of the app is terminated. Then the background tasks associated with this app are cancelled. The state of the app is checked, and if the app is still running then it will fail this test.

Corrective action

Add the cancellation handler to your app. For more information see [Support your app with background tasks](#).

App count

This verifies that an app package (.msix, .appx, or app bundle) contains one application. This was changed in the kit to be a standalone test.

Background

This test was implemented as per Store policy.

Test details

For Windows Phone 8.1 apps the test verifies the total number of .appx packages in the bundle is < 512, there is only one main package in the bundle, and that the architecture of the main package in the bundle is marked as Arm or neutral.

For Windows 10 apps the test verifies that the revision number in the version of the bundle is set to 0.

Corrective action

Ensure the app package and bundle meet requirements above in Test details.

App manifest compliance test

Test the contents of app manifest to make sure its contents are correct.

Background

Apps must have a correctly formatted app manifest.

Test details

Examines the app manifest to verify the contents are correct as described in the [App package requirements](#).

- **File extensions and protocols**

Your app can declare the file extensions that it wants to associate with. Used improperly, an app can declare a large number of file extensions, most of which it may not even use, resulting in a bad user experience. This test will add a check to limit the number of file extensions that an app can associate with.

- **Framework Dependency rule**

This test enforces the requirement that apps take appropriate dependencies on the UWP. If there is an inappropriate dependency, this test will fail.

If there is a mismatch between the OS version the app applies to and the framework dependencies made, the test will fail. The test would also fail if the app refers to any preview versions of the framework dlls.

- **Inter-process Communication (IPC) verification**

This test enforces the requirement that UWP apps do not communicate outside of the app container to Desktop components. Inter-process communication is intended for side-loaded apps only. Apps that specify the [ActivatableClassAttribute](#) with name equal to "DesktopApplicationPath" will fail this test.

Corrective action

Review the app's manifest against the requirements described in the [App package requirements](#).

Windows Security features test

Background

Changing the default Windows security protections can put customers at increased risk.

Test details

Tests the app's security by running the [BinScope Binary Analyzer](#).

The BinScope Binary Analyzer tests examine the app's binary files to check for coding and building practices that make the app less vulnerable to attack or to being used as an attack vector.

The BinScope Binary Analyzer tests check for the correct use of the following security-related features.

- BinScope Binary Analyzer tests
- Private Code Signing

BinScope Binary Analyzer tests

The [BinScope Binary Analyzer](#) [↗] tests examine the app's binary files to check for coding and building practices that make the app less vulnerable to attack or to being used as an attack vector.

The BinScope Binary Analyzer tests check for the correct use of these security-related features:

- [AllowPartiallyTrustedCallersAttribute](#)
- [/SafeSEH Exception Handling Protection](#)
- [Data Execution Prevention](#)
- [Address Space Layout Randomization](#)
- [Read/Write Shared PE Section](#)
- [AppContainerCheck](#)
- [ExecutableImportsCheck](#)
- [WXCheck](#)

AllowPartiallyTrustedCallersAttribute

Windows App Certification Kit error message: APTCACHeck Test failed